CMPS4232 -Test #2
Note: Preparatory Task
Due: Provided by the Lecturer

1. We have been tasked with writing a bash shell script that assists a Go developer in setting up a directory structure to develop their API project. Your script must accomplish the following:

   a. The script should be run by the user as shown below. The user must provide two arguments to the script. The first is the name of the top-level directory , and the second is the identifier. In our example, we use `karpbox` as the name of the top-level directory, and `lewisdalwin.net` is the identifier for the go.mod file

   `make-go-dir  karpbox  umana-amilcar.net`

   Note: If the user does not provide exactly two arguments, you need to prompt them to do so. For example:

   `You need to provide two arguments for this script to work.`
   `Usage: make-go-dir <root directory name> <identifier name>`




```
/bin/bash

Check if exactly two arguments were provided
[ "$#" -ne 2 ]; then
  echo "You need to provide two arguments for this script to work."
  echo "Usage: make-go-dir <root directory name> <identifier name>"
  exit 1

Assign arguments to variables
OT_DIR=$1
ENTIFIER=$2

Ask the user for confirmation
ho "I am about to create a directory structure named '$ROOT_DIR'."
ad -p "Do you want me to continue? [Yes/no] " RESPONSE

Check user's response
[[ "$RESPONSE" == "Yes" || "$RESPONSE" == "yes" ]]; then

  echo "Create directory structure..."

  # Create the directory structure
  mkdir -p "$ROOT_DIR"/{bin,cmd/api,internals,migrations,remote}

  # Create main.go file

lp       ^O Write Out   ^F Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set
```

b. Your script should prompt the user to verify if they want to continue to create the directory structure shown below.

`I am about to create a directory structure named 'karpbox'.`
`Do you want me to continue? [Yes/no]`

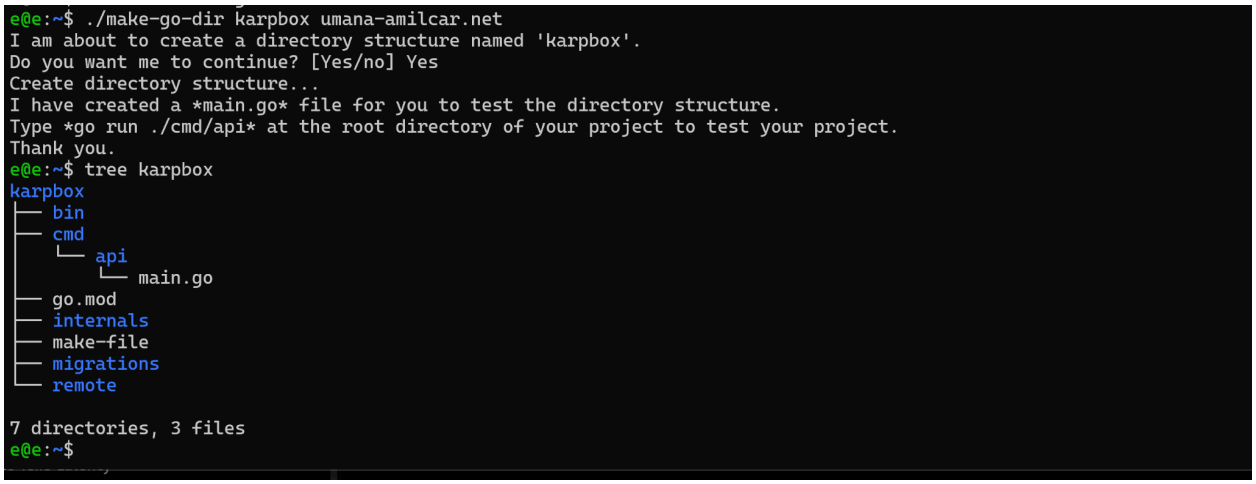
c. If the user types `Yes`, then proceed with the directory structure creation.

`Create directory structure...`

d. If the user types `no`, then print an abort message.

`Abort.`

e. Below is the directory structure. `main.go`, `go.mod`, and `make-file` are all files. The others are directories with the '.' representing the root directory of the project (the name that the user typed in. In our example, `karpbox`.



.

|-- bin
|-- cmd
| |___ api
| |___ main.go
|-- internals
|-- migrations
|-- remote
|-- go.mod
|-- make-file

f. Your script should write a `hello world` program to the `main.go` file as shown below.

```
// File: cmd/api/main.go

package main

import "fmt"

func main() {
fmt.Println("Hello world!")
}
```

g. Your script should add the identifier to the `go.mod` file. In our example `karpbox.umana-amilcar.net`

```
e@e:~/karpbox$ cat go.mod
module karpbox.umana-amilcar.net
e@e:~/karpbox$
```

h. Finally, you should write a message to the user telling them that a `main.go` file has been created for them to test their project, as shown below.

```
`I have created a *main.go* file for you to test the directory structure.`
`Type *go run ./cmd/api* at the root directory of your project to test your project.`
'Thank you.`
```

```
No VM guests are running outdated hypervisor (qemu) binari
e@e:~/karpbox$ go run ./cmd/api
Hello world!
```

created